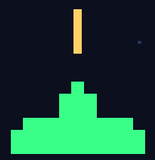
REINFORCEMENT LEARNING · PROJECT PROPOSAL

Space Invaders from Scratch

Training Rainbow DQN in a Custom, Instrumented Environment

Team: Sergey Peshkov · Anton Kropotov · Arina Nikolaeva

Innopolis University · October 8, 2026



MOTIVATION

Why Space Invaders?

- **Classic RL benchmark** — Atari Space Invaders is one of the standard games used to evaluate DQN-family agents since 2013–2015.
- **Rich but tractable** — Moving enemies, projectiles, destructible shields and delayed rewards — hard enough to be interesting, small enough to train on a student budget.
- **Black-box problem** — With the original ROM (ALE) we only see pixels and score — we cannot change rules, rewards or inspect the game state.
- **Our idea** — Re-implement the game ourselves so we fully control what the agent sees, what it is rewarded for, and how hard the game is.

OBJECTIVE

Project Goal & Research Questions

Goal

Build a Space Invaders clone with a Gymnasium-compatible API and configurable reward "triggers", then train a Rainbow DQN agent and study how environment design affects learning.

RQ1 Which Rainbow components matter most in our game? (ablation study)

RQ2 How do shaped reward triggers change sample efficiency and final behaviour?

RQ3 Does a difficulty curriculum speed up training compared to the full game from the start?

RQ4 Pixels vs. compact state vector: how big is the gap?

APPROACH

Why Build the Game Ourselves?

Atari ROM (ALE)

- Fixed rules and score-only reward
- Only pixels / RAM as observations
- No access to internal events
- Fixed difficulty

Our implementation

- Event hooks ("triggers") for any game event
- Both pixel and state-vector observations
- Tunable difficulty & curriculum
- Fast headless mode for training
- Human-playable mode for demo

MDP FORMULATION

Environment Design

State S

- Pixels: 84×84 grayscale, stack of 4 frames
- OR vector: player x, invader grid, bullets, shields, lives
- Fully Markov with frame stacking

Actions A

- 6 discrete actions:
- NOOP, LEFT, RIGHT, FIRE, LEFT+FIRE, RIGHT+FIRE
- Frame-skip k = 4

Reward R

- Base: game score (10/20/30, UFO bonus)
- Optional shaped triggers (next slide)
- Rewards clipped / scaled for stability

Episode ends when the player loses all lives or invaders reach the ground. Gymnasium API: reset() / step(a).

WHAT THE CUSTOM GAME GIVES US

Reward Triggers & Difficulty Knobs

Event triggers (reward shaping)

`enemy_killed`	+1 × enemy value
`ufo_killed`	+ bonus
`player_hit`	– penalty
`shot_missed`	small – (ammo efficiency)
`dodged_bullet`	small +
`wave_cleared`	+ large
`time_step`	–ε (anti-stalling)

Difficulty / curriculum knobs

- Invader speed & fire rate
- Number of rows / columns
- Shields on / off
- Player lives
- Random seeds for generalisation tests

ALGORITHM

Rainbow DQN = DQN + 6 Improvements

Double Q-learning

Removes overestimation bias by decoupling action selection and evaluation.

Prioritized Replay

Samples transitions with high TD-error more often.

Dueling Network

Separate value V(s) and advantage A(s,a) streams.

Multi-step returns

n-step targets propagate rewards faster (n = 3).

Distributional (C51)

Learns the return distribution over 51 atoms, not just its mean.

Noisy Nets

Learned parametric noise replaces ε-greedy exploration.

Hessel et al., 2018 — combining all six gives the strongest Atari results of the DQN family.

MODEL

Network Architecture

Output: Q-value distribution Z(s,a) over 51 atoms for each of the 6 actions;
action = argmax of expected value. Vector-state variant swaps the conv stack for an MLP.

METHODOLOGY

Experimental Plan

Baselines & main runs

- Random agent & human play (reference)
- Vanilla DQN
- Full Rainbow DQN
- Pixels vs. state-vector input
- 3 random seeds per configuration

Ablations & studies

- Rainbow minus one component (×6)
- Score-only reward vs. shaped triggers
- Curriculum vs. full difficulty
- Generalisation to unseen difficulty settings

HOW WE MEASURE SUCCESS

Evaluation Metrics

- **Mean episode score** — averaged over 30 evaluation episodes, no exploration noise
- **Sample efficiency** — frames needed to reach a target score
- **Human-normalised score** — (agent – random) / (human – random)
- **Behaviour stats** — accuracy (hits / shots), survival time, waves cleared
- **Training stability** — variance across seeds, learning curves

TOOLS

Tech Stack

Game	Python + Pygame, headless rendering for fast simulation
Env API	Gymnasium interface, vectorised environments
Learning	Adapted open-source PyTorch Rainbow DQN
Tracking	TensorBoard / Weights & Biases, config files per run
Compute	Single GPU; target budget ~5–10M frames per main run
Demo	Live game: human vs. trained agent, with trigger overlay

PLAN

Timeline

Exam on Nov 12 is a buffer week — long training runs keep going in the background.

WHO DOES WHAT

Team & Responsibilities

Arina Nikolaeva

ALGORITHM & EXPLANATION

- Study Rainbow DQN and its six components
- Write the algorithm explanation in the shared notebook
- Step-by-step worked example of one update with calculations
- Algorithm slides + her part of the talk

Anton Kropotov

CODE & TRAINING

- Find and adapt an existing Rainbow DQN implementation
- Connect the environment, set up runs
- Train the agent (or use a suitable pretrained model)
- Save model, hyperparameters, training results
- Notebook code to load and run the model
- Implementation slides + his part of the talk

Sergey Peshkov

ENVIRONMENT, RESULTS & DEMO

- Define the environment: states, actions, rewards, goal
- Environment description + evaluation code in notebook
- Compare trained agent vs. random over several runs
- Results plot / table and demo recording
- Environment & results slides
- Assemble the final 5–7 min video

PLANNING AHEAD

Risks & Mitigations

Slow training	→ headless fast mode, vector-state variant, frame-skip, parallel envs
Bugs in Rainbow	→ test each component separately on CartPole before Space Invaders
Reward hacking via triggers	→ always report score-only metric; inspect agent videos
Game too easy / too hard	→ difficulty knobs + calibrate on human and random players
Limited GPU time	→ fix frame budget early; prioritise main runs over ablations

DELIVERABLES

Expected Outcomes

- **Open-source game** — Space Invaders clone with Gymnasium API and configurable triggers.
- **Rainbow DQN agent** — Trained agent that clearly beats random and DQN baselines.
- **Analysis** — Learning curves, ablation table, reward-shaping and curriculum study.
- **Live demo** — Agent playing in real time, with event triggers visualised on screen.
- **Notebook & video** — Shared Colab notebook (explanation, worked example, evaluation) and a 5–7 min video.

THANK YOU · QUESTIONS?

References

Mnih et al. (2015). Human-level control through deep reinforcement learning. Nature.

van Hasselt, Guez, Silver (2016). Deep RL with Double Q-learning. AAAI.

Schaul et al. (2016). Prioritized Experience Replay. ICLR.

Wang et al. (2016). Dueling Network Architectures for Deep RL. ICML.

Bellemare, Dabney, Munos (2017). A Distributional Perspective on RL. ICML.

Fortunato et al. (2018). Noisy Networks for Exploration. ICLR.

Hessel et al. (2018). Rainbow: Combining Improvements in Deep RL. AAAI.

Ng, Harada, Russell (1999). Policy invariance under reward transformations. ICML.